

EXPERIMENT – 1

Familiarization of environments in R.

R provides an environment for statistical computing, data analysis, graphical representation, and programming. When RStudio is opened, it provides a graphical interface through which we can write and execute R programs.

The major components we need to become familiar with are:

1. R Console
2. Script Editor / Source Pane
3. Environment Pane
4. History Pane
5. Files Pane
6. Plots Pane

1.1: R Console: The Console is where R commands are directly entered and executed. The Console is useful for:

1. Executing commands immediately
2. Testing small pieces of code
3. Performing calculations
4. Checking functions
5. Viewing output

For larger programs, however, you should generally use the Script Editor.

1.2: Script Editor (Source Pane): The Script Editor is where you write and save R programs.

Create a new R script using: File → New File → R Script

You can also use: Ctrl + Shift + N

Save the file as: <filename>.R

The .R extension indicates an R source-code file.

You can execute individual lines or selected code from the script.

Why use a script? - Unlike typing everything directly into the Console, your code can be: Saved, Edited, Reused, Shared

1.3: Environment Pane: A graphical interface in RStudio that displays the objects currently available in the active R environment, along with information about their names, values, and types.

When you execute:

```
x <- 1
```

R creates an object called x.

Now look at the Environment pane.

You should see something similar to:

Environment

```
x    10
```

The Environment pane therefore provides a convenient view of the objects currently available in your R session.

1.4: History Pane: The History pane records commands that you have previously executed.

For example, if you run:

```
x <- 10
y <- 20
x + y
mean(c(10,20,30))
```

these commands will appear in the History.

This is useful when you:

- Forget a command you previously used
- Want to reuse an earlier command
- Want to inspect your recent work

1.6: Files Pane: The Files pane allows you to browse files and directories from within RStudio.

For example, you might have:

```
STUR/
├── Experiment1.R
├── Experiment2.R
├── data.csv
└── output/
```

You can navigate through these files directly from RStudio.

This becomes particularly useful when your practicals start involving datasets.

1.7: Plots Pane: The Plots pane displays graphs generated using R.

For example, execute:

```
x <- c(10, 20, 30, 40, 50)
plot(x)
```

R will generate a graph, which should appear in the Plots pane.

The Plots pane allows you to view and, depending on the plot, export the generated graph.

1.8: Sample Program

```
age <- 21.5
cat("Numeric:", age, "\n")
marks <- 85L
cat("Integer:", marks, "\n")
name <- "Abhi"
cat("Character:", name, "\n")
is_student <- TRUE
cat("Logical:", is_student, "\n")
z <- 3 + 2i
cat("Complex:", z, "\n")
numbers <- c(10, 20, 30, 40)
cat("Numeric Vector:", numbers, "\n")
names <- c("Abhi", "Rahul", "Aman")
cat("Character Vector:", names, "\n")
```

Output:

The screenshot shows the RStudio IDE with a script editor on the left and a console on the bottom. The script contains the following R code:

```

6
7 name <- "Abhi"
8 cat("Character:", name, "\n")
9
10 is_student <- TRUE
11 cat("Logical:", is_student, "\n")
12
13 z <- 3 + 2i
14 cat("Complex:", z, "\n")
15
16 numbers <- c(10, 20, 30, 40)
17 cat("Numeric Vector:", numbers, "\n")
18
19 names <- c("Abhi", "Rahul", "Aman")
20 cat("Character Vector:", names, "\n")
21

```

The console output shows the execution of these commands:

```

> age <- 21.5
> cat("Numeric:", age, "\n")
Numeric: 21.5
> marks <- 85L
> cat("Integer:", marks, "\n")
Integer: 85
> name <- "Abhi"
> cat("Character:", name, "\n")
Character: Abhi
> is_student <- TRUE
> cat("Logical:", is_student, "\n")
Logical: TRUE
> z <- 3 + 2i
> cat("Complex:", z, "\n")
Complex: 3+2i
> numbers <- c(10, 20, 30, 40)
> cat("Numeric Vector:", numbers, "\n")
Numeric Vector: 10 20 30 40
> names <- c("Abhi", "Rahul", "Aman")
> cat("Character Vector:", names, "\n")
Character Vector: Abhi Rahul Aman
>

```

The screenshot shows the RStudio console with the following output:

```

R - R 4.6.1 - ~/
> age <- 21.5
> cat("Numeric:", age, "\n")
Numeric: 21.5
> marks <- 85L
> cat("Integer:", marks, "\n")
Integer: 85
> name <- "Abhi"
> cat("Character:", name, "\n")
Character: Abhi
> is_student <- TRUE
> cat("Logical:", is_student, "\n")
Logical: TRUE
> z <- 3 + 2i
> cat("Complex:", z, "\n")
Complex: 3+2i
> numbers <- c(10, 20, 30, 40)
> cat("Numeric Vector:", numbers, "\n")
Numeric Vector: 10 20 30 40
> names <- c("Abhi", "Rahul", "Aman")
> cat("Character Vector:", names, "\n")
Character Vector: Abhi Rahul Aman
>

```

The screenshot shows the RStudio Environment pane with the following objects:

Name	Type	Length	Size	Value
age	numeric	1	56 B	21.5
is_student	logical	1	56 B	TRUE
marks	integer	1	56 B	85L
name	character	1	112 B	"Abhi"
names	character	3	248 B	chr [1:3] "Abhi" "Rahul" "Ama..."
numbers	numeric	4	80 B	num [1:4] 10 20 30 40
z	complex	1	64 B	3+2i

EXPERIMENT – 2

Perform simple arithmetic's using R.

Code:

```
a <- 20
b <- 5

print("Addition:")
print(a + b)

print("Subtraction:")
print(a - b)

print("Multiplication:")
print(a * b)

print("Division:")
print(a / b)

print("Modulus:")
print(a %% b)

print("Integer Division:")
print(a %/% b)

print("Power:")
print(a ^ b)
```

Output:

```
> source("~/5th Sem/Practical Programs/R/2.R")
[1] "Addition:"
[1] 25
[1] "Subtraction:"
[1] 15
[1] "Multiplication:"
[1] 100
[1] "Division:"
[1] 4
[1] "Modulus:"
[1] 0
[1] "Integer Division:"
[1] 4
[1] "Power:"
[1] 3200000
```

EXPERIMENT – 3

Perform basic R functions.

Code:

```
numbers <- c(10, 20, 30, 40, 50)

# Basic mathematical functions
cat("Sum:", sum(numbers), "\n")
cat("Mean:", mean(numbers), "\n")
cat("Minimum:", min(numbers), "\n")
cat("Maximum:", max(numbers), "\n")
cat("Range:", range(numbers), "\n")
cat("Length:", length(numbers), "\n")

# Mathematical functions
cat("Absolute value:", abs(-25), "\n")
cat("Square root:", sqrt(25), "\n")
cat("Round:", round(12.5678, 2), "\n")
cat("Ceiling:", ceiling(12.3), "\n")
cat("Floor:", floor(12.8), "\n")

# Statistical functions
cat("Median:", median(numbers), "\n")
cat("Standard deviation:", sd(numbers), "\n")
cat("Variance:", var(numbers), "\n")

# Sorting functions
cat("Sorted values:", sort(numbers), "\n")
cat("Reverse sorted values:", sort(numbers, decreasing = TRUE), "\n")

# Sequence function
cat("Sequence:", seq(1, 10, by = 2), "\n")

# Repetition function
cat("Repeat:", rep(5, 4), "\n")

# Type checking functions
x <- 10
cat("Is numeric:", is.numeric(x), "\n")
cat("Is character:", is.character(x), "\n")
cat("Is integer:", is.integer(x), "\n")

# Conversion functions
y <- "100"
cat("Character to numeric:", as.numeric(y), "\n")

# Other useful functions
cat("Class:", class(numbers), "\n")
cat("Number of elements:", length(numbers), "\n")
```

Output:

```
> source("~/5th Sem/Practical Programs/R/3.R")
Sum: 150
Mean: 30
Minimum: 10
Maximum: 50
Range: 10 50
Length: 5
Absolute value: 25
Square root: 5
Round: 12.57
Ceiling: 13
Floor: 12
Median: 30
Standard deviation: 15.81139
Variance: 250
Sorted values: 10 20 30 40 50
Reverse sorted values: 50 40 30 20 10
Sequence: 1 3 5 7 9
Repeat: 5 5 5 5
Is numeric: TRUE
Is character: FALSE
Is integer: FALSE
Character to numeric: 100
Class: numeric
Number of elements: 5
```

EXPERIMENT – 4

Use various graphical techniques in EDA.

Code:

```
# Create sample data
students <- data.frame(
  Name = c("Abhi", "Saurav", "Mahira", "Shyam", "Shweta", "Akash", "Ibrahim", "Pritam"),
  Marks = c(85, 75, 90, 65, 80, 87, 23, 45),
  Hours = c(5, 3, 6, 2, 4, 3, 1, 2)
)
print(students)

# 1. Histogram
hist(
  students$Marks,
  main = "Histogram of Marks",
  xlab = "Marks"
)

# 2. Boxplot
boxplot(
  students$Marks,
  main = "Boxplot of Marks",
  ylab = "Marks"
)

# 3. Bar Plot
barplot(
  students$Marks,
  names.arg = students$Name,
  main = "Marks of Students",
  xlab = "Students",
  ylab = "Marks"
)

# 4. Pie Chart
pie(students$Marks,
  labels = students$Name,
  main = "Marks Distribution"
)

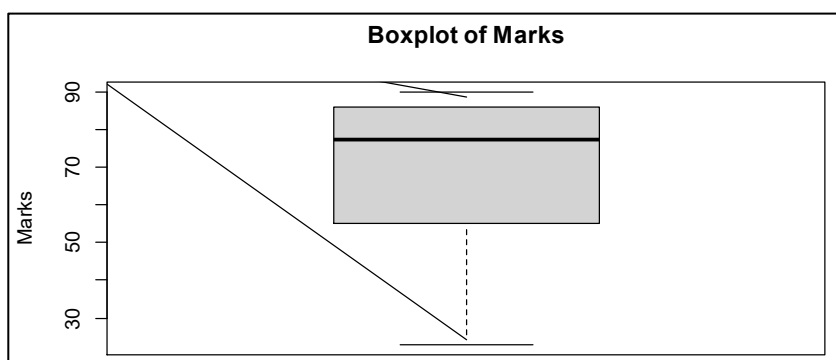
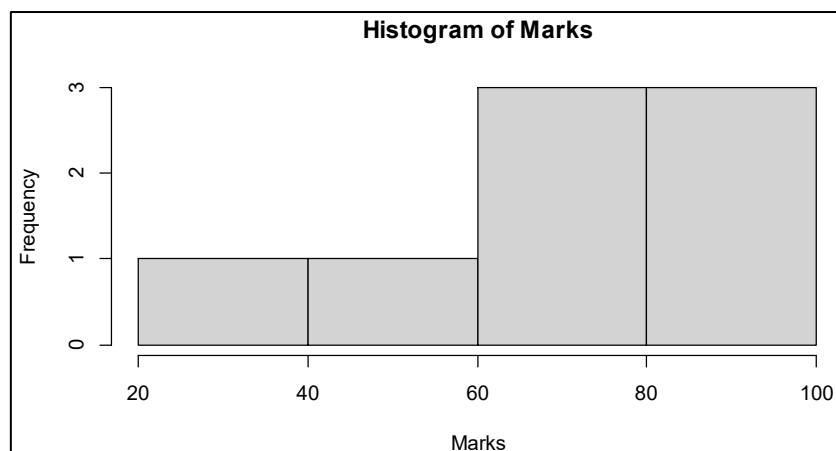
# 5. Scatter Plot
plot(
  students$Hours,
  students$Marks,
  main = "Study Hours vs Marks",
  xlab = "Study Hours",
  ylab = "Marks"
)
```

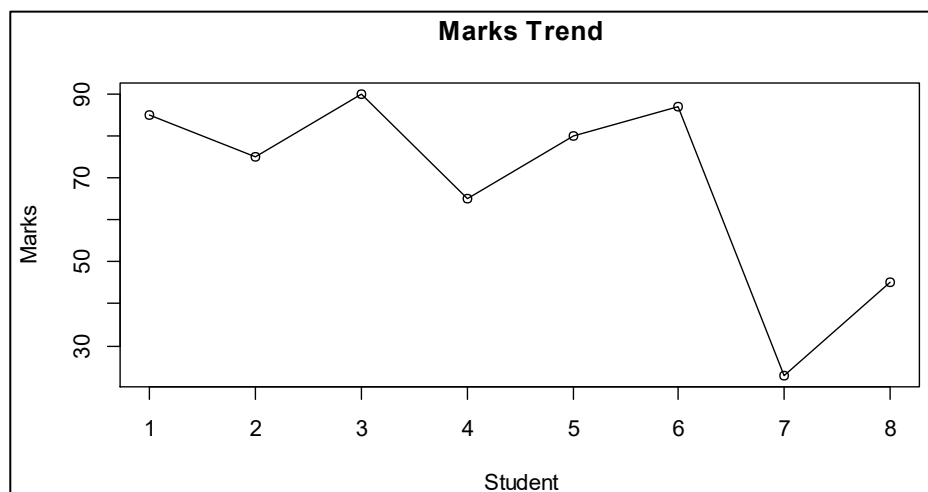
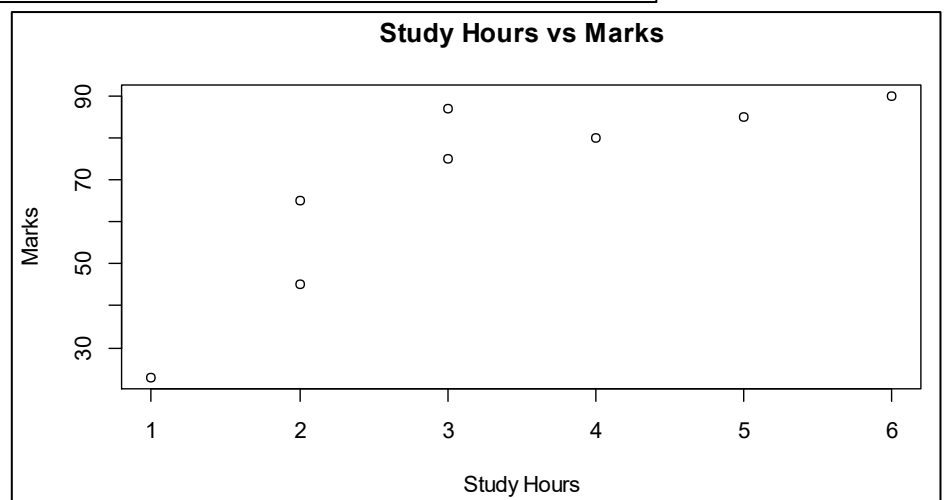
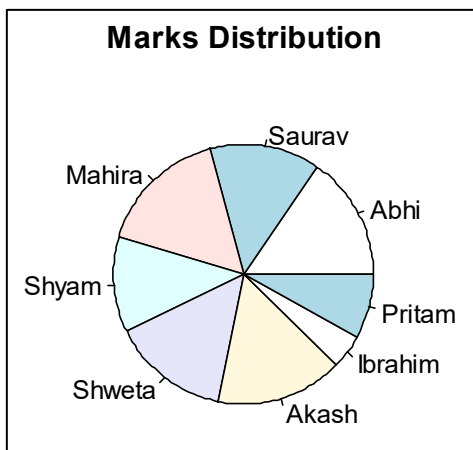
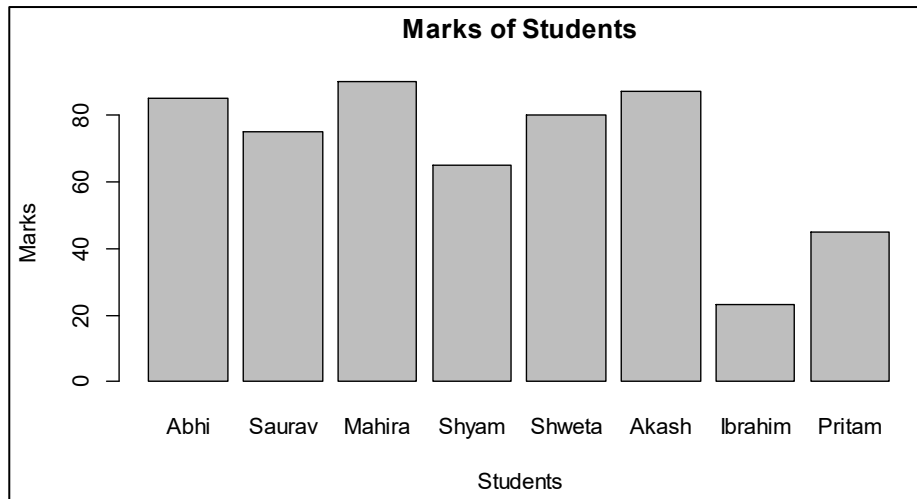
6. Line Plot

```
plot(  
  students$Marks,  
  type = "o",  
  main = "Marks Trend",  
  xlab = "Student",  
  ylab = "Marks"  
)
```

Output:

```
> source("~/5th Sem/Practical Programs/R/4.R")  
      Name Marks Hours  
1    Abhi    85     5  
2  Saurav    75     3  
3  Mahira    90     6  
4   Shyam    65     2  
5  Shweta    80     4  
6   Akash    87     3  
7 Ibrahim    23     1  
8  Pritam    45     2
```

Plots Obtained:



EXPERIMENT – 5

Create different charts for visualization of given set of data.

Code:

```
# Load the built-in dataset
data(mtcars)

# Display the dataset
print(head(mtcars))

# 1. Bar Chart - Number of Cars by Cylinders
cylinder_count <- table(mtcars$cyl)

barplot(
  cylinder_count,
  names.arg = names(cylinder_count),
  main = "Cars by Number of Cylinders",
  xlab = "Number of Cylinders",
  ylab = "Number of Cars"
)

# 2. Histogram - Miles per Gallon
hist(
  mtcars$mpg,
  main = "Distribution of MPG",
  xlab = "Miles per Gallon",
  ylab = "Frequency"
)

# 3. Boxplot - MPG
boxplot(
  mtcars$mpg,
  main = "Boxplot of MPG",
  ylab = "Miles per Gallon"
)

# 4. Scatter Plot - MPG vs Weight
plot(
  mtcars$wt,
  mtcars$mpg,
  main = "MPG vs Weight",
  xlab = "Weight",
  ylab = "Miles per Gallon",
  pch = 19
)

# 5. Pie Chart - Transmission Type
pie(
  table(mtcars$am),
```

```

labels = c("Automatic", "Manual"),
main = "Transmission Types"
)

```

6. Line Chart - MPG of Cars

```

plot(
  mtcars$mpg,
  type = "o",
  main = "MPG of Cars",
  xlab = "Car Number",
  ylab = "Miles per Gallon"
)

```

Output:

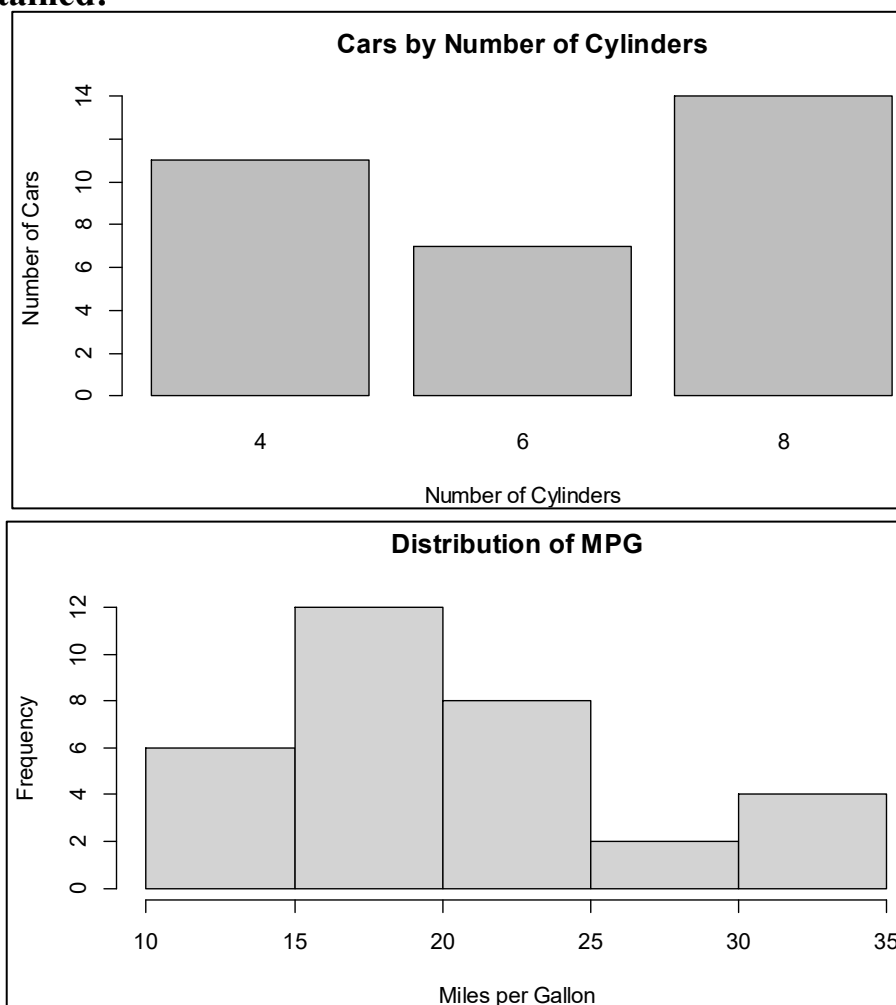
```

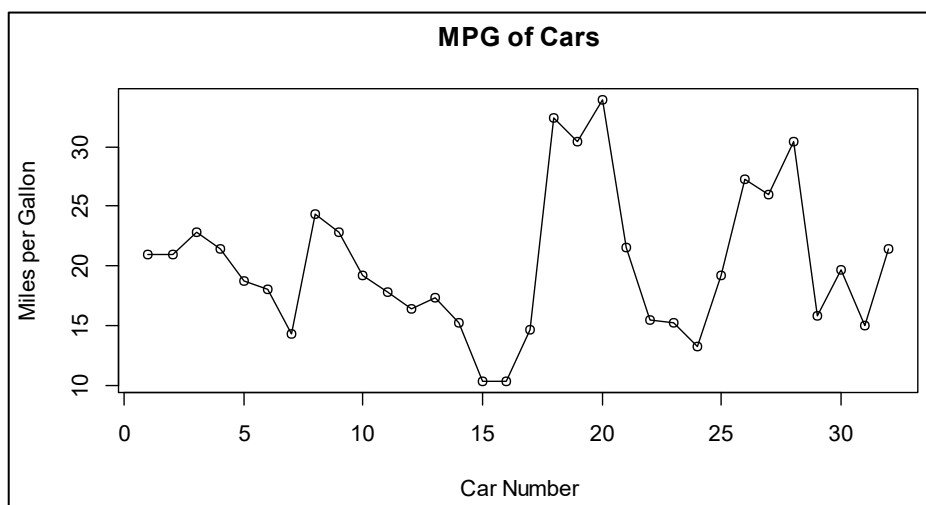
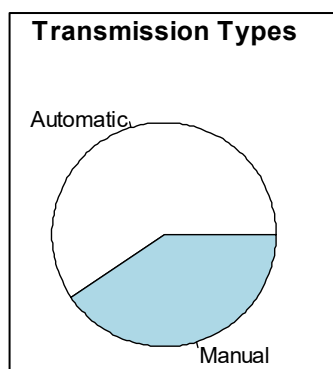
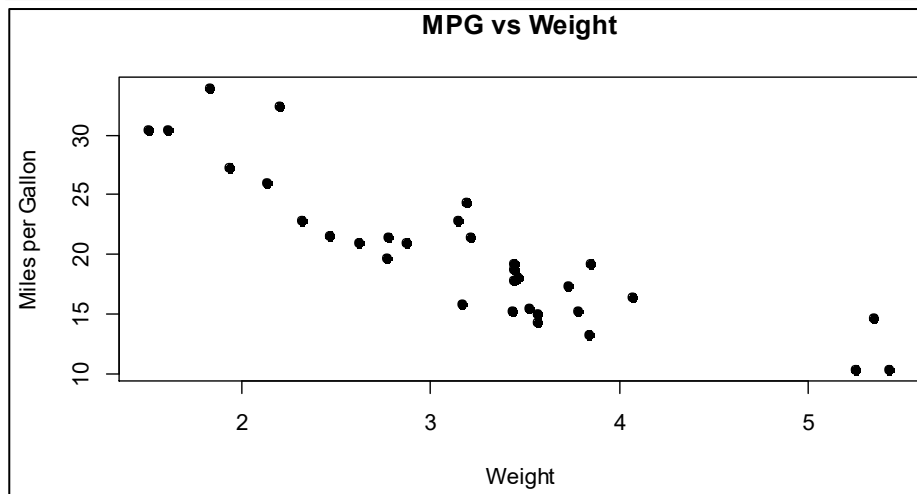
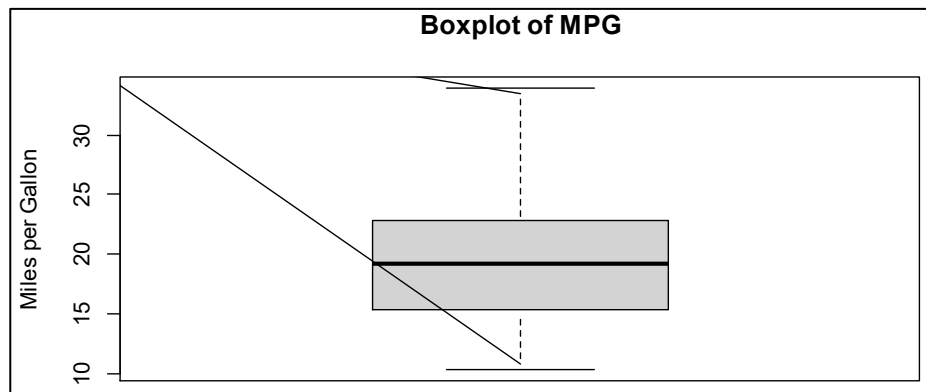
> source("~/5th Sem/Practical Programs/R/5.R")

```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

Plots Obtained:





EXPERIMENT – 6

Find the mean, median, standard deviation and quartiles of a set of observations.

Code:

```
x <- c(10, 80, 93, 56, 36, 6, 93, 72, 62, 43)

mean_value <- mean(x)
median_value <- median(x)
sd_value <- sd(x)
quartiles <- quantile(x)

cat("Given set of observations:", x, "\n\n")

cat("Number of Observations: ", length(x), "\n")
cat("Mean: ", mean(x), "\n")
cat("Median: ", median(x), "\n")
cat("Standard Deviation: ", sd(x), "\n")
cat("Quartiles:\n")
print(quartiles)
```

Output:

```
> source("~/5th Sem/Practical Programs/R/6.R")
Given set of observations: 10 80 93 56 36 6 93 72 62 43

Number of Observations: 10
Mean: 55.1
Median: 59
Standard Deviation: 31.23904
Quartiles:
  0%   25%   50%   75%  100%
6.00 37.75 59.00 78.00 93.00
```

EXPERIMENT – 7**Find the Skewness and Kurtosis of a given dataset distribution.****Code:**

```
library(moments)

data(iris)
cat("6 TOPMOST ROWS OF DATASET:\n")
print(head(iris))

skewness_values <- apply(iris[, 1:4], 2, skewness)
kurtosis_values <- apply(iris[, 1:4], 2, kurtosis)
cat("\nSKEWNESS VALUES OF ALL NUMERIC COLUMNS:\n")
print(skewness_values)
cat("\nKURTOSIS VALUES OF ALL NUMERIC COLUMNS:\n")
print(kurtosis_values)

hist(
  iris$Sepal.Length,
  main = "Distribution of Sepal Length with KDE",
  xlab = "Sepal Length",
  ylab = "Density",
  breaks = 20,
  border = "darkgreen",
  col = "lightgreen",
  ylim = c(0, 0.6),
  xlim = c(4, 8),
  freq = FALSE
)

kde_Sepal.Length <- density(iris$Sepal.Length)

lines(
  kde_Sepal.Length,
  col = "red",
  lwd = 2
)

hist(
  iris$Sepal.Width,
  main = "Distribution of Sepal Width with KDE",
  xlab = "Sepal Width",
  ylab = "Density",
  breaks = 20,
  border = "darkgreen",
  col = "lightgreen",
  ylim = c(0, 1.9),
  xlim = c(2, 4.5),
  freq = FALSE
)
```

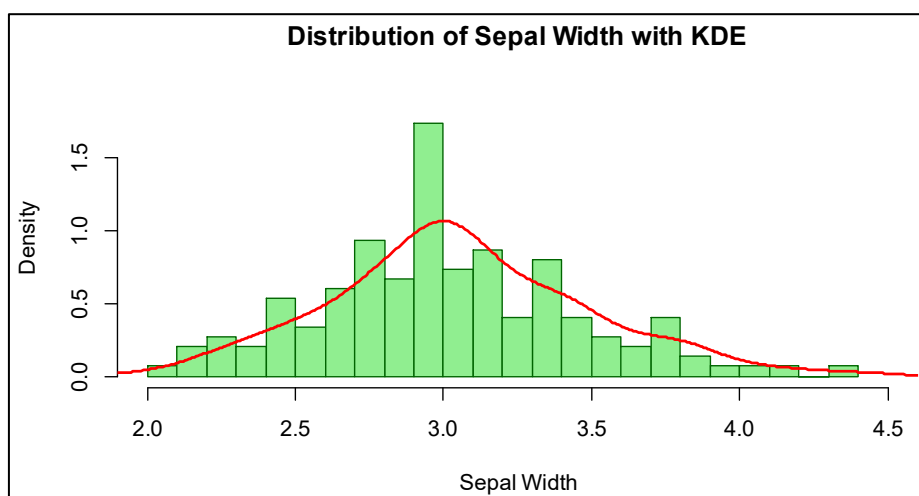
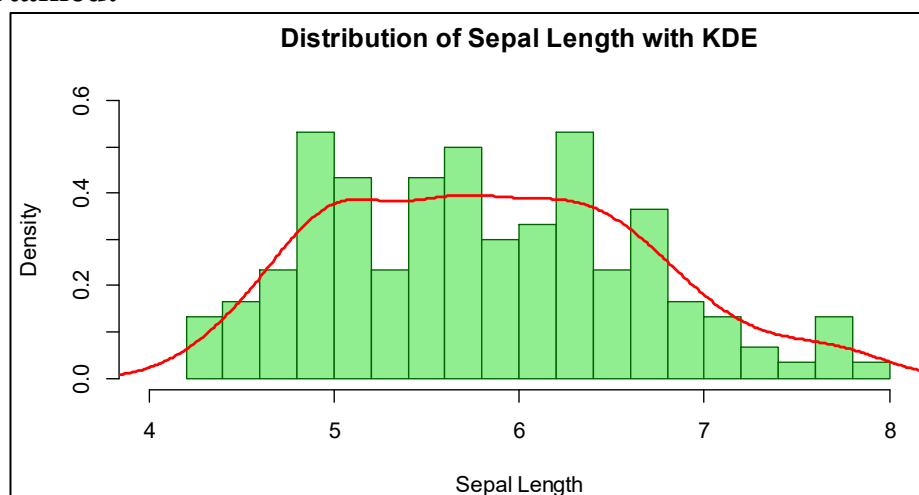
```
)  
  
kde_Sepal.Width <- density(iris$Sepal.Width)  
  
lines(  
  kde_Sepal.Width,  
  col = "red",  
  lwd = 2  
)  
  
hist(  
  iris$Petal.Length,  
  main = "Distribution of Petal Length with KDE",  
  xlab = "Petal length",  
  ylab = "Density",  
  breaks = 20,  
  border = "darkgreen",  
  col = "lightgreen",  
  ylim = c(0, 0.6),  
  xlim = c(1, 6.9),  
  freq = FALSE  
)  
  
kde_Petal.Length <- density(iris$Petal.Length)  
  
lines(  
  kde_Petal.Length,  
  col = "red",  
  lwd = 2  
)  
  
hist(  
  iris$Petal.Width,  
  main = "Distribution of Petal Width with KDE",  
  xlab = "Petal Width",  
  ylab = "Density",  
  breaks = 20,  
  border = "darkgreen",  
  col = "lightgreen",  
  ylim = c(0, 2.4),  
  xlim = c(0.1, 2.6),  
  freq = FALSE  
)  
  
kde_Petal_Width <- density(iris$Petal.Width)  
lines(  
  kde_Petal_Width,  
  col = "red",  
  lwd = 2  
)
```

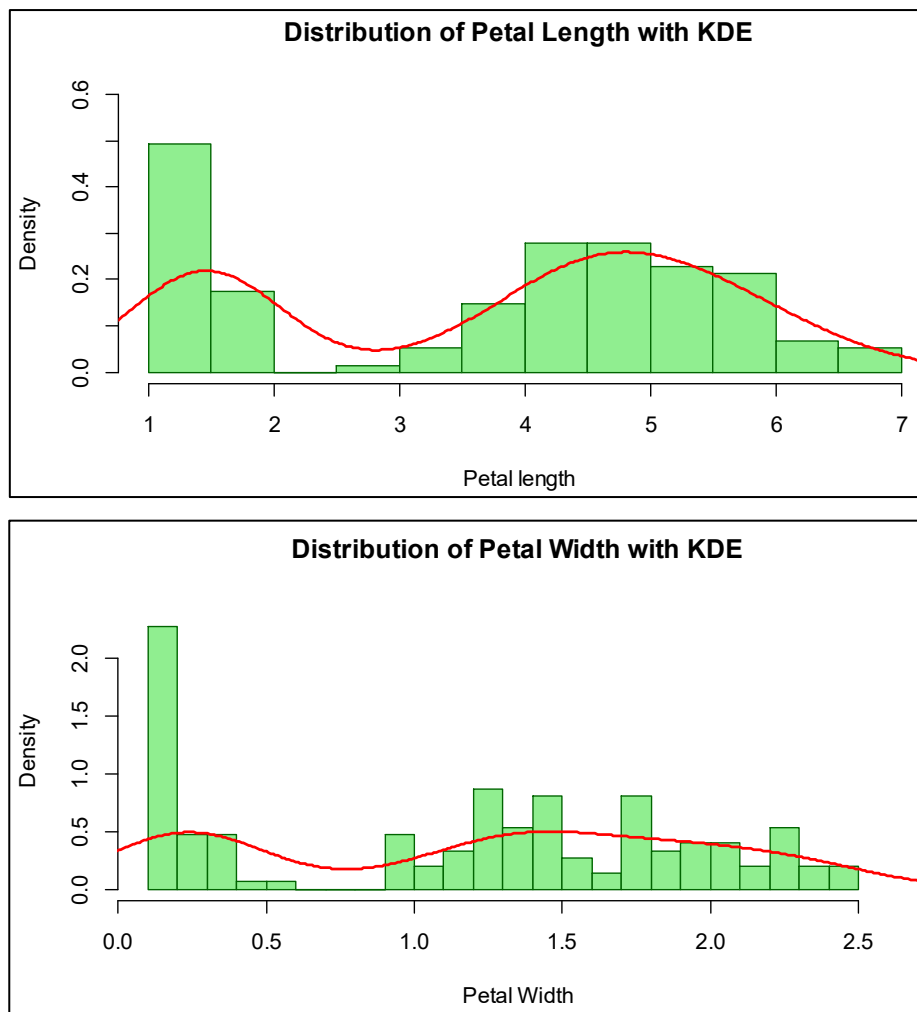
Output:

```
6 TOPMOST ROWS OF DATASET:
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1          3.5          1.4          0.2  setosa
2          4.9          3.0          1.4          0.2  setosa
3          4.7          3.2          1.3          0.2  setosa
4          4.6          3.1          1.5          0.2  setosa
5          5.0          3.6          1.4          0.2  setosa
6          5.4          3.9          1.7          0.4  setosa
```

```
SKEWNESS VALUES OF ALL NUMERIC COLUMNS:
Sepal.Length Sepal.Width Petal.Length Petal.Width
  0.3117531   0.3157671  -0.2721277  -0.1019342
```

```
KURTOSIS VALUES OF ALL NUMERIC COLUMNS:
Sepal.Length Sepal.Width Petal.Length Petal.Width
  2.426432    3.180976    1.604464    1.663933
```

Plots Obtained:



EXPERIMENT – 8

Given the scenario, implement the Bayes rule by finding the posterior probability.

Code: